Adversarial Fraud & AML Detection Arena

A MINOR PROJECT REPORT SUBMITTED TO

THE NATIONAL INSTITUTE OF ENGINEERING, MYSURU
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

in partial fulfillment for the award of degree of

Bachelor of Engineering
in
Computer Science and Engineering

Submitted By

Anirudh Krishna 4NI24CS022
Golla Venkata Aditya 4NI24CS067
Gourav R 4NI24CS068
Akshay MP 4NI24CS010

Under the guidance of

Dr. B R Vatsala
Associate Professor,
Department of CS&E
NIE, Mysuru

Department of Computer Science & Engineering
THE NATIONAL INSTITUTE OF ENGINEERING
(Autonomous Institution)
Mysuru - 570 008

2026-27

THE NATIONAL INSTITUTE OF ENGINEERING
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the project work entitled "**Adversarial Fraud & AML Detection Arena**" is a bonafide work carried out by Anirudh Krishna (4NI24CS022)**,** Golla Venkata Aditya (4NI24CS067)**,** Gourav R (4NI24CS068)**,** Akshay MP (4NI24CS010) in partial fulfillment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering**, of Visvesvaraya Technological University, Belagavi, during the year **2026-27**. It is certified that all corrections / suggestions indicated during internal assessment have been incorporated and the corrected copy has been deposited in the department library. This project report has been approved in partial fulfillment for the award of the said degree as per academic regulations of The National Institute of Engineering (Autonomous Institution).

Dr. B R Vatsala
Associate Professor
Dept. of CSE
NIE, Mysuru

Dr. Anitha R
Professor and Head
Dept. of CS&E
NIE, Mysuru

Name of the Examiners **Signature with Date**

1. _____ _____

ABSTRACT

Fraud and anti-money-laundering (AML) detectors are normally validated once, on a fixed dataset of patterns that were already known when the model was trained. Real launderers adapt, so such a score says little about robustness. This project builds an Adversarial Fraud & AML Detection Arena: a closed-loop system in which a gradient-boosted (LightGBM) detector is attacked by a genetic-algorithm adversary that evolves realistic laundering strategies, namely structuring amounts below the reporting threshold, spacing transactions in time and routing funds through intermediate accounts, with a fitness that rewards both evasion and completion of the laundering goal. The strategies the adversary discovers are fed back into the detector's training data, and the two are retrained against each other for several rounds, producing a measurable arms-race curve of detector recall against adversary success.

Four invariants keep the measurement honest: the adversary emits transactions and never features, money is conserved by construction, the decision threshold comes from a fixed alert budget, and the clean holdout never sees adversary data. The system was developed on a synthetic transaction world and then run unchanged on the IBM AML benchmark (HI-Small, 5.08 million transactions). On IBM data the detector reaches a recall of 0.668 at a 0.5% alert budget, yet even a hand-written laundering strategy goes almost unflagged, because the detector largely learned that laundering is paid by ACH, and five rounds of retraining did not close the gap. On synthetic data, retraining raised the detector's recall against its contemporary adversary from 0.125 to 0.510. A Streamlit dashboard presents the transaction flows, evolved strategies and attribution drift round by round. The project also shows that an adversary is an effective auditor: every near-perfect evasion score exposed a flaw in the data, the simulation or the experiment itself.

Keywords: anti-money laundering, adversarial machine learning, genetic algorithm, LightGBM, arms race, IBM AML dataset.

ACKNOWLEDGMENT

We sincerely owe our gratitude to all the persons who helped and guided us in completing this mini project.

We are thankful to **Dr. B. S. Nagendra Parashar**, Principal, NIE College, Mysuru, for all the support she has rendered.

We thank **Dr. Anitha R**, Professor and Head, Department of Computer Science and Engineering, for her constant support and encouragement throughout the tenure for the mini project work.

We would like to sincerely thank our guide **Dr. B R Vatsala, Associate Professor,** Department of Computer Science and Engineering, for providing relevant information, valuable guidance and encouragement to complete this minor project.

Anirudh Krishna 4NI24CS022

Golla Venkata Aditya 4NI24CS067

Gourav R 4NI24CS068

Akshay MP 4NI24CS010

TABLE OF CONTENTS

CONTENTS

Abstract	I
Acknowledgement	II
Table Of Contents	III
List Of Figures	IV
1. Introduction	**1**
1.1 Background	1
1.2 Purpose	2
1.3 Objectives	2
2. Literature Review	**3**
3. Existing System and Proposed System	**5**
3.1 Existing System	5
3.2 Proposed System	5
4. System Requirements	**7**
4.1 Hardware Requirements	7
4.2 Software Requirements	7
5. System Design	**8**
5.1 System Architecture	8
5.2 Use Case Diagram	11
6. Implementation	**14**
6.1 Pseudocode	14
6.2 Implementation Details	15
7. Testing	**17**
7.1 Testing Strategy	17
7.2 Automated Test Suite	17
7.3 Experimental Evaluation	17
7.4 Defects Found by the Adversary and the Experiments	26
7.5 Discussion	27
8. Screenshots	**28**
Future Enhancement and Conclusion	**35**
References	**36**

LIST OF FIGURES

FIG. NO.	DESCRIPTION	PAGE NO.
5.1	System architecture of the Adversarial Fraud & AML Detection Arena	8
5.2	One round of the closed loop and what is measured after it	11
5.3	Use case diagram	12
7.1	IBM HI-Small: precision-recall curve on the frozen holdout, with the 0.5% operating point	18
7.2	IBM HI-Small: recall at the alert budget by typology	19
7.3	Synthetic data: detector recall and adversary success over five rounds	20
7.4	Synthetic data: cross-play matrix, every detector against every adversary	21
7.5	Mean absolute TreeSHAP attribution per feature on the adversary's transactions, detectors D_0 to D_5	22
7.6	Fixed projection of real transactions coloured by each detector's decisions	23
7.7	Cross-dataset transfer (left) and feature-group ablation (right)	24
7.8	IBM HI-Small: detector recall and adversary success over five rounds	25
7.9	IBM HI-Small: cross-play matrix	26
8.1	Overview page: the headline result and the four invariants	28
8.2	Phase 1 page: the detector and why accuracy is not used	29
8.3	Phase 2 page: genetic-algorithm convergence and the naive versus evolved gap	29
8.4	Phase 3 page: the arms-race curve with bootstrap intervals	30
8.5	Phase 3 page: cross-play matrix and the cost on the clean holdout	30
8.6	Phase 4 page: one round, one operation: decoded transfers scored by D_r and $D_{(r+1)}$	31
8.7	Phase 4 page: attribution drift per feature per detector	31
8.8	Phase 5 page: IBM data cleaning, found by profiling the dataset first	32
8.9	Phase 5 page: the detector on real data, recall by typology	32
8.10	Phase 5 page: transfer between the synthetic and the IBM world	33
8.11	Phase 5 page: why the adversary wins on IBM data, and the arms race beside the synthetic one	33
8.12	Phase 5 page: IBM holdout cost and the cross-play matrix	34
8.13	Live explorer: set the twelve genes and score the decoded transactions	34

Chapter 1

INTRODUCTION

1.1 Background

Money laundering is the process of disguising the proceeds of crime so that they appear to come from a legitimate source. It is usually described in three stages: placement, where illegal cash enters the financial system; layering, where the money is moved through many transactions and accounts to hide its origin; and integration, where it re-emerges as apparently clean wealth. Banks are legally required to monitor transactions and report suspicious activity, and cash-type transactions above a reporting threshold (for example $10,000 in the United States) trigger mandatory reports. Launderers therefore split amounts into smaller pieces that stay just below such thresholds, a practice called structuring.

Automated transaction monitoring began as hand-written rules and has moved towards machine-learning models that rank transactions so that a limited team of analysts can review the most suspicious ones. Two properties make this problem unlike ordinary classification. The first is **extreme class imbalance**: in the IBM AML dataset used in this project only about one transaction in a thousand (0.10%) is laundering, so a model that predicts "not laundering" for everything is 99.9% accurate and completely useless. The second is an **adaptive opponent**: launderers change their behaviour in response to detection, so a detector that scores well on last year's patterns says little about next year's.

The laundering patterns that detectors are trained to recognise are called typologies. The five used in this project are:

- **Fan-out:** one account sends money to many accounts.
- **Fan-in:** many accounts send money to one account.
- **Cycle:** money passes through a ring of accounts and returns to its origin.
- **Gather-scatter:** money is gathered into one account and then scattered to many.
- **Stack:** money is layered through a sequence of intermediate accounts.

The usual way of evaluating such a detector is to train it on a labelled dataset and report a score on a held-out part of the same dataset. That measures how well the detector recognises the patterns it has already been shown. It does not measure how it behaves against a launderer who is actively trying to evade it, which is the situation in which a detector is actually used.

1.2 Purpose

The purpose of this project is to replace the single static evaluation of an AML detector with an **arms race**. A detector is placed in a closed loop with a genetic-algorithm adversary that evolves evasive laundering strategies against it; the adversary's discoveries are fed back into the detector's training data; and the two are retrained and re-measured over successive rounds. The result is not one accuracy figure but a set of curves and matrices that show whether the detector is becoming harder to fool, whether it forgets earlier lessons, and what each improvement costs on ordinary laundering.

A second purpose is methodological. An evaluation that includes an optimiser searching for weaknesses will exploit any flaw in the data or the simulation just as readily as a real weakness in the detector. The project therefore treats a suspiciously good adversarial score as a defect to be investigated, and records every flaw found this way (Chapter 7).

1.3 Objectives

The objectives set in the project synopsis are:

1. To design and train a fraud/AML transaction classifier on the IBM AML dataset, capable of recognising the known laundering typologies: fan-out, fan-in, cycle and gather-scatter transfers.
2. To build a genetic-algorithm-based adversarial agent whose genome encodes realistic evasion actions (structuring, timing delays and intermediate-account routing), constrained to plausible transaction behaviour.
3. To define a fitness function balancing detection evasion against completion of the adversary's laundering objective, and to evolve evasive strategies across successive generations.
4. To implement a closed feedback loop in which evasive strategies are reincorporated into the detector's training data, and to measure detector recall and adversary success rate across multiple retraining rounds.
5. To build a visualisation dashboard presenting transaction flows, evolved evasion strategies and the detector's response, demonstrating the arms race as an observable, round-by-round process.

The work was done in phases (scaffold, detector, adversary, closed loop, visualisation, real IBM data), each producing metrics, figures, tests and a written account. The system was first developed on a synthetic transaction generator, where ground truth is known and the pipeline can be debugged cheaply, and was then run unchanged on the IBM AML benchmark.

Chapter 2

LITERATURE REVIEW

Fraud detection and evaluation. Bolton and Hand [1] reviewed the statistical approaches to fraud detection and distinguished supervised methods, which learn from labelled fraud, from unsupervised methods that look for unusual behaviour. Dal Pozzolo et al. [2] studied credit-card fraud detection under realistic conditions and stressed that investigators can inspect only a limited number of alerts, so a detector is better judged on its highest-ranked alerts than on global accuracy. This motivates the fixed alert budget used here: the detector may flag only 0.5% of transactions, and recall is measured at that budget.

Imbalanced data and metrics. Saito and Rehmsmeier [3] showed that the precision-recall plot is more informative than the ROC plot on imbalanced data, because the false-positive rate barely moves when there are hundreds of negatives per positive. This project therefore reports recall at the alert budget and PR-AUC, and labels ROC-AUC as misleading. Dal Pozzolo et al. [4] showed that undersampling the majority class distorts predicted probabilities, which is harmless here because the decision threshold is chosen as a score quantile and only the ranking of scores matters.

Gradient-boosted trees. Chen and Guestrin [5] (XGBoost) and Ke et al. [6] (LightGBM) made gradient-boosted decision trees fast and accurate on large tabular data. They handle skewed features and mixed numerical and categorical inputs without much preprocessing, train in seconds on millions of rows, and score a few dozen rows in microseconds, which matters because the adversary's genetic algorithm must score tens of thousands of candidates. LightGBM was chosen for the detector.

Graph-based AML. Weber et al. [7] applied graph convolutional networks to a labelled Bitcoin transaction graph and compared them with conventional classifiers, illustrating that the relationships between accounts carry much of the signal. This project captures the same signal with hand-engineered account-graph features (degrees, counterparty counts, burst ratios, cycle indicators) feeding a tree model. That is cheaper to evaluate inside an optimisation loop and, through the TreeSHAP method [14], can be explained exactly.

Datasets. Real labelled laundering data cannot be published, so public benchmarks are simulated. PaySim [8] simulates mobile-money transactions for fraud detection. Altman et al. [9] built the AMLworld generator, a multi-agent simulation of a banking world with injected laundering patterns (fan-out, fan-in, cycle, gather-scatter, scatter-gather, bipartite, stack and random), and released it as the IBM AML dataset [16] in several sizes and laundering-rate variants. The IBM dataset is the primary

data source of this project; PaySim has no comparable account graph or typology labels and was not used.

Adversarial machine learning. Goodfellow et al. [10] showed that small, deliberate input perturbations can make accurate classifiers fail, and Biggio and Roli [11] surveyed a decade of adversarial machine learning and argued that security-sensitive classifiers must be evaluated against adaptive attackers. Most of that work perturbs continuous inputs such as images. In transaction monitoring an attacker cannot edit features; the attacker can only create transactions, which must obey accounting rules. This is why the adversary here emits transactions and never features, and why money conservation is built into the decoder.

Evolutionary search. Genetic algorithms [12] search a space of candidate solutions by selection, crossover and mutation, and need only a score for each candidate, not a gradient. That suits an adversary whose genome mixes integers and fractions and whose fitness comes from running a black-box detector. DEAP [13] provides the evolutionary machinery used here. Scikit-learn [15] supplies the preprocessing and principal component analysis used in the visualisation.

Table 2.1: Related work and the gap this project addresses

Work	Contribution	Gap addressed by this project
Bolton and Hand [1]	Review of statistical fraud detection	Evaluation against a fixed pattern set, not an adaptive opponent
Dal Pozzolo et al. [2]	Realistic fraud evaluation; limited alert capacity	Capacity is adopted (0.5% budget); the adversary is added
Chen and Guestrin [5]; Ke et al. [6]	Fast gradient-boosted trees	Used as the detector; robustness to evasion is measured
Weber et al. [7]	Graph neural networks for AML on Bitcoin	Account-graph features used instead; evaluated adversarially
Altman et al. [9]	AMLworld generator and IBM AML datasets	Used as the real-data benchmark; shortcut analysis added
Goodfellow et al. [10]; Biggio and Roli [11]	Adversarial examples; security evaluation of classifiers	Attacker restricted to valid, money-conserving transactions
Lundberg et al. [14]	Exact TreeSHAP for tree ensembles	Used to show attribution drift between rounds

In summary, the literature provides the detector technology, the evaluation metrics, the benchmark datasets, the adversarial viewpoint and the search method. What it does not provide, and what this project adds, is a closed loop in which a transaction-level adversary and an AML detector are retrained against each other, measured with a fixed alert budget on a frozen clean holdout, and audited for the dataset and simulation artefacts that such an adversary tends to expose.

Chapter 3

EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

In the existing practice, an AML or fraud detector is built in four steps: a labelled historical dataset is collected, a model is trained on the earlier part of it, the model is scored on a later part, and a single figure (often accuracy or ROC-AUC) is reported. Banks combine such models with fixed rules, for example alerts on amounts just below the reporting threshold. The model is retrained periodically on newly labelled cases, which arrive slowly because laundering is confirmed only after investigation.

This arrangement has the following weaknesses:

- **It validates only against known patterns.** The score describes how well the model recognises laundering like the laundering it was trained on, not laundering that has been adapted to avoid it.
- **Its headline metrics mislead at this prevalence.** Accuracy is about 99.9% for a model that predicts nothing, and ROC-AUC stays high even when the alert queue is unusable.
- **It cannot show robustness.** There is no measurement of how much recall falls when the opponent changes strategy, nor of whether retraining on new strategies makes the detector forget old ones.
- **It hides dataset shortcuts.** If a benchmark contains an artefact, such as one payment format that marks laundering, the score rewards learning the artefact.
- **Rules are easy to probe.** A fixed threshold rule is trivially evaded by structuring just beneath it.

3.2 Proposed System

The proposed system is the **Adversarial Fraud & AML Detection Arena**, a closed loop of a detector and an adversary. A LightGBM detector is trained on labelled transactions. A genetic-algorithm adversary evolves populations of laundering strategies whose genes encode structuring, timing and routing; each strategy is decoded into real transactions and scored by the detector through the same feature pipeline that built the training data. Fitness rewards strategies that complete the laundering goal while staying unflagged. The most evasive distinct strategies are folded back into the detector's training set, under guards that stop the detector from collapsing, forgetting or overfitting, and the detector is retrained. Repeating this for several rounds yields an arms-race curve, a cross-play matrix of every detector against every adversary, and the cost of robustness on a frozen clean holdout.

The system's main advantages are:

- Robustness is **measured against an adapting opponent**, round by round, instead of asserted from one score.
- The adversary can only act through valid transactions, so an evasion is an evasion a real launderer could attempt, and money is conserved by construction.
- The metrics are honest at 0.1% prevalence: recall at a fixed 0.5% alert budget and PR-AUC.
- The loop doubles as an **audit tool**: an adversary that reaches an implausibly perfect score has found a flaw in the data or simulation, and the decoded transactions show where.
- The same unchanged pipeline runs on synthetic and on real IBM data, so the two can be compared directly.
- Everything is observable through a dashboard that replays each round's flows, strategy and detector response.

Table 3.1: Existing practice compared with the proposed system

Aspect	Existing practice	Proposed arena
Evaluation	One static train/test split	Repeated rounds against an adapting adversary
Headline metric	Accuracy or ROC-AUC	Recall at a fixed 0.5% alert budget, PR-AUC
Opponent	None; known patterns only	Genetic-algorithm adversary evolving evasive strategies
Threshold	Tuned freely	Fixed policy: score quantile spending the alert budget on the clean holdout
Retraining	Periodic, on newly labelled cases	On the adversary's strategies, with caps, slices and time windows
Robustness evidence	None	Arms-race curve, cross-play matrix, confidence intervals
Dataset artefacts	Rewarded silently	Surfaced by the adversary, then fixed structurally
Explanation	Feature importance at best	Exact TreeSHAP attribution drift and decoded transaction flows

Chapter 4

SYSTEM REQUIREMENTS

4.1 Hardware Requirements

The synopsis set the following hardware requirements. The right-hand column records the machine on which the project was actually developed and run.

Table 4.1: Hardware requirements

Item	Minimum (synopsis)	Used in development
Processor	Intel Core i5 / AMD Ryzen 5 or higher	12 logical cores
Memory (RAM)	8 GB minimum, 16 GB recommended	32 GB (largest run: about 4.3 GB per process)
Storage	15 GB free, SSD recommended	Under 2 GB for data, models and reports
Network	Active internet connection	Needed only to download the dataset and packages
Operating system	Windows 10/11, Linux (Ubuntu 20.04+) or macOS	Windows 11

Run times on this machine give an idea of the compute needed. On the IBM data, building the account graph takes about 46 seconds, training the detector about 28 seconds, evolving one adversary (20,500 candidate evaluations) about 3 minutes, and the five-round closed loop about 25 minutes. On the 2-million-row synthetic ledger the corresponding figures are a 6-minute adversary run and a 29-minute closed loop. No GPU is required.

4.2 Software Requirements

Table 4.2: Software requirements

Component	Choice and version	Purpose
Language	Python 3.12 (pinned; 3.14 avoided because of wheel availability)	All code
Environment	uv	Reproducible virtual environment and command runner
Detector	LightGBM 4.7	Gradient-boosted classifier
Machine learning	scikit-learn 1.9	Scaling, PCA, metrics support
Adversarial engine	DEAP 1.4, NumPy	Genetic algorithm; vectorised numerics
Data handling	pandas 3.0, PyArrow	Ledgers and Parquet storage
Dashboard	Streamlit 1.64, Plotly, Matplotlib	Interactive visualisation; report figures
Testing	pytest	102 automated tests
Notebooks, version control	Jupyter; Git and GitHub	One notebook per phase; history and delivery
Data	IBM AML dataset (HI-Small); synthetic generator	Training and evaluation; PaySim was not used

Chapter 5

SYSTEM DESIGN

5.1 System Architecture

The system has five layers (Fig. 5.1). The **data layer** produces a canonical, validated ledger from either the synthetic generator or the IBM loader. The **feature layer** holds a frozen graph of the background ledger, a copy-on-write overlay for candidate laundering transactions, and the single `featurize()` function used by both. The **players** are the detector and the adversary. The **loop layer** orchestrates rounds, retraining and evaluation. The **output layer** writes JSON metrics and figures that the dashboard reads.

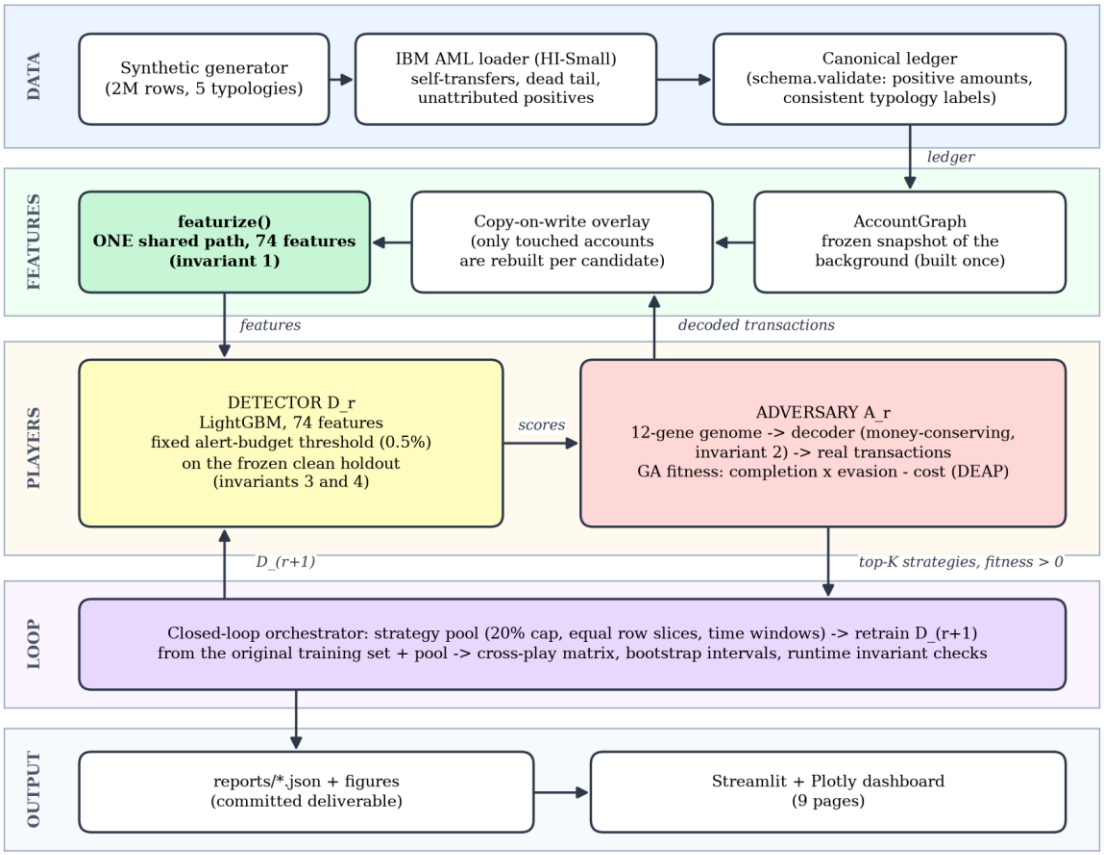


Fig. 5.1: System architecture of the Adversarial Fraud & AML Detection Arena

Canonical ledger

Every data source is normalised into one schema before anything else touches it, and `validate()` rejects ledgers with non-positive amounts, self-transfers, or typology labels inconsistent with the laundering flag. This stops a plausible-looking but impossible transaction, in particular one produced by the adversary's decoder, from reaching the detector.

Table 5.1: The canonical ledger schema

Field	Meaning
timestamp	Time of the transaction
from_bank, from_account	Sending bank and account
to_bank, to_account	Receiving bank and account
amount_paid, payment_currency	Amount debited and its currency
amount_received, receiving_currency	Amount credited and its currency
payment_format	ACH, cheque, credit card, reinvestment, wire, cash or bitcoin
is_laundering	Ground-truth label (0 or 1)
typology	Fan-out, fan-in, cycle, gather-scatter, stack, or unattributed (real data only)
pattern_id	Groups the transactions of one laundering instance
source	background or adversary: lets a clean holdout be proven clean

Feature engineering

Each transaction is described by 74 features, all computed as of the transaction's own timestamp so that no future information leaks into the past. There are **17 per-transaction features** (log amount, low-order digits, distance to the $10,000 threshold, hour, weekday, cross-bank and cross-currency flags, FX loss, payment format) and **57 relational features** for the sender and the receiver: counts, volumes and distinct counterparties over 1-hour, 1-day and 7-day windows, burst ratios, flow ratios, account age, time since the previous transaction, repeated-pair counts, a gather-scatter score and 2-, 3- and 4-hop cycle indicators.

Detector and threshold policy

The detector is a LightGBM classifier trained on all positives and 25 negatives per positive, with the positive class weighted by the observed imbalance and early stopping on validation PR-AUC. The decision threshold is **not tuned**: it is the score that flags exactly 0.5% of the transactions of the frozen clean holdout, mirroring a fixed analyst capacity. Because the same policy is applied to every detector, recall is comparable across rounds.

Adversary genome and decoder

The genome has twelve genes (Table 5.2). It encodes only **relative** quantities: how many tranches, how unevenly, how close to the threshold. It never encodes absolute amounts. The decoder turns a genome and an episode (move a goal amount from a source account to a destination account by a deadline, using real background accounts as intermediaries) into a list of ordinary transactions, and always allocates exactly the goal amount. Consequently any genome, including any crossover offspring, conserves money.

Table 5.2: The twelve-gene adversary genome

Gene	Range	Meaning
typology_template	0 to 3	Fan-out, cycle, gather-scatter or stack routing
n_hops	1 to 4	Routing depth through intermediate accounts
n_splits	1 to 12	Structuring: tranches per leg
split_skew	0 to 1	0 equal tranches, 1 highly uneven
threshold_hug	0 to 1	How close tranches sit below $10,000
mean_delay_h	0 to 72 h	Mean delay between transactions
delay_jitter	0 to 1	Variation of the delays
business_hours_bias	0 to 1	Probability of landing in 09:00 to 17:00
mule_pool_size	1 to 10	Intermediate accounts available
mule_reuse	0 to 1	0 a fresh account per tranche, 1 reuse one
format_mix	0 to 1	0 low-scrutiny formats, 1 high-scrutiny formats
dormancy_h	0 to 168 h	Hold time at each intermediate hop

Fitness

A strategy's fitness is $\text{completion}^{\alpha} \times \text{evasion}^{\beta} - \gamma \times \text{cost}$, with $\alpha = \beta = 1$ and $\gamma = 0.1$. Completion is the share of the goal that reaches the destination by the deadline; evasion is one minus the share of transferred value that the detector flags; cost combines fees and elapsed time. If less than 95% of the money arrives the fitness is exactly zero, so a strategy cannot score well by moving only part of it. Each candidate is averaged over five episodes so the search cannot overfit one lucky account.

The closed loop

One round (Fig. 5.2) evolves an adversary against the current detector, keeps up to eight mutually distinct strategies with positive fitness, decodes each on training-period episodes, and retrains the next detector from the **original** training set plus the pool. Retraining from the original set, not from the previous round's augmented set, prevents drift from compounding. Further guards limit the feedback to 20% of the original positives, split into equal row slices per round so no earlier lesson is traded away, and confine each operation to its own time window.

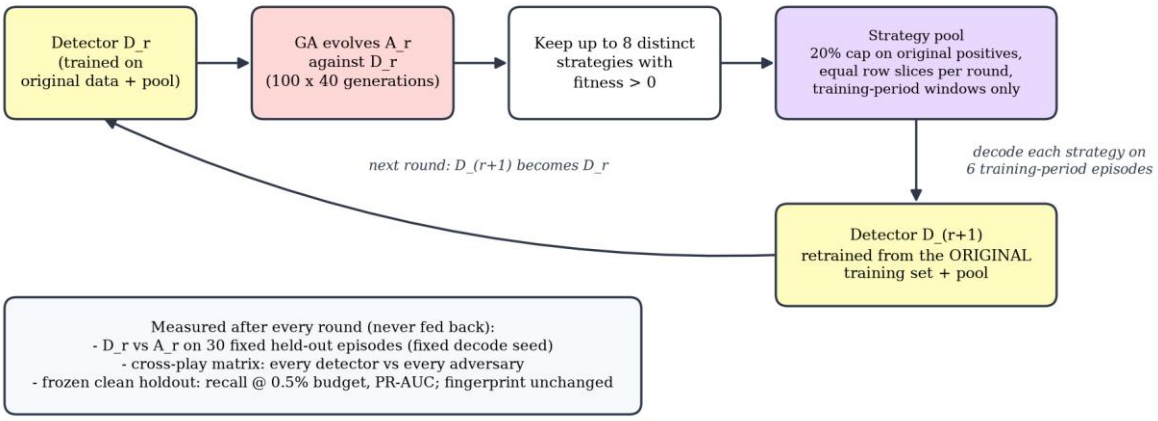


Fig. 5.2: One round of the closed loop and what is measured after it

The four invariants

1. **The adversary emits transactions, never features.** One `featurize()` serves both training and the adversary's evaluation, and a test checks that the batch path and the overlay path agree.
2. **Money conservation is a decoder invariant,** not a repair step, and is tested on random crossover offspring and, for real data, on random genomes.
3. **The threshold comes from a fixed policy** (0.5% alert budget on the clean holdout) and is never re-tuned per round.
4. **The clean holdout never sees adversary data.** It is carved out chronologically before the adversary exists and its fingerprint is verified unchanged at the end of every run.

Software structure

Table 5.3: Module map

Module	Responsibility
schema.py, config.py	Canonical ledger contract; YAML configuration with an overlay mechanism
data/	Synthetic generator, IBM loader and cleaning, single entry point load_ledger()
features/	AccountGraph (frozen plus overlay), rolling windows, the shared featurize()
detector/	LightGBM wrapper, time-based split, alert-budget threshold, metrics
adversary/	Genome, decoder, constraints, fitness, DEAP driver
loop/	Strategy pool, evaluation, orchestrator, re-evaluation with confidence intervals, gate
explain.py	TreeSHAP attribution drift, fixed projection, decoded flows
scripts/	One command-line script per step
dashboard/	Streamlit application with nine pages

5.2 Use Case Diagram

Fig. 5.3 shows the use cases of the system. The primary actor is the researcher or student who prepares

data, trains the detector, evolves strategies, runs the arms race and reads the results. The dataset source (the IBM files or the synthetic generator) supplies the transactions. A guide or examiner uses the dashboard to view results.

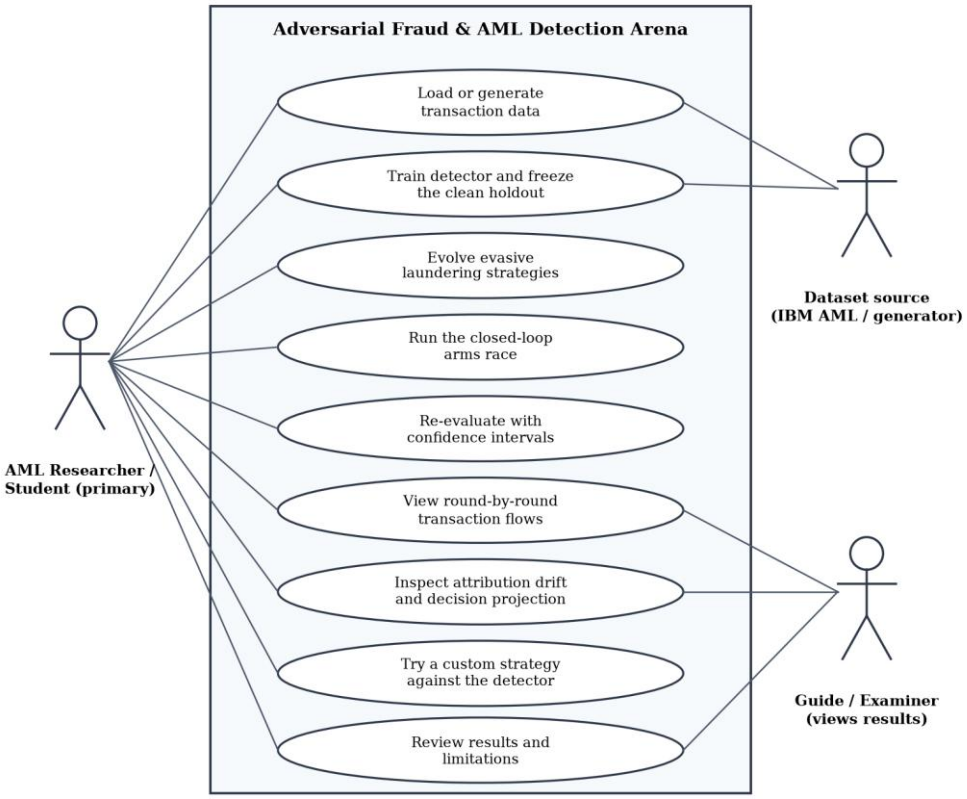


Fig. 5.3: Use case diagram

Table 5.4: Use case descriptions

Use case	Actor	Description
Load or generate transaction data	Researcher, dataset source	Build a validated canonical ledger from the generator or the IBM files, with cleaning
Train detector and freeze the holdout	Researcher	Chronological split, LightGBM training, fixed alert-budget threshold
Evolve evasive strategies	Researcher	Run the genetic algorithm against a detector and report the champion
Run the closed-loop arms race	Researcher	Rounds of evolve, feed back, retrain; cross-play matrix
Re-evaluate with confidence intervals	Researcher	Re-measure a finished run on 200 episodes with bootstrap intervals
View round-by-round transaction flows	Researcher, examiner	Draw a champion's decoded transfers scored by two detectors
Inspect attribution drift and projection	Researcher, examiner	TreeSHAP per feature per detector; fixed PCA plane of decisions
Try a custom strategy	Researcher	Set the twelve genes by slider and score the decoded transactions live
Review results and limitations	Researcher, examiner	Read the headline results and the stated caveats

Chapter 6

IMPLEMENTATION

6.1 Pseudocode

The pseudocode below summarises the core algorithms. Names follow the source code in `src/arena/`.

Algorithm 1: Alert-budget threshold and the shared feature path

```
FUNCTION alert_threshold(scores_on_clean_holdout, budget = 0.005):
    RETURN the (1 - budget) quantile of scores_on_clean_holdout
    # fixed policy: never re-tuned per round; recall is measured at this threshold

FUNCTION featurize(transactions, graph, windows):          # the ONLY feature path
    FOR each transaction t:
        FOR each of sender, receiver:
            counts, volumes, distinct counterparties, bursts over windows
                computed from events strictly BEFORE t.timestamp     # as-of rule
        add per-transaction features (amount, threshold gap, hour, format, ...)
        add cycle flags of length 2..4 and the gather-scatter score
    RETURN feature matrix
```

Algorithm 2: Copy-on-write overlay

```
FUNCTION overlay(graph, sub_ledger):
    touched <- accounts appearing in sub_ledger
    new_state[a] <- merge(graph.state[a], events of sub_ledger for a)  for a in touched
    RETURN view that reads new state first and falls through to graph for all other accounts
    # cost grows with the number of touched accounts, not with the size of the ledger
```

Algorithm 3: Money-conserving decoder

```
FUNCTION decode(genome, episode, rng):
    p <- decode_genes(genome);  layers <- route(p.template, p.n_hops, src, dst, mules)
    holdings <- { src: episode.goal }
    FOR each hop layer:
        FOR each sender with balance >= MIN_TRANCHE:
            tranches <- split(balance, n_splits, split_skew, threshold_hug)   # sums to balance
            legs <- currency legs from the world's own process (see Algorithm 6)
            t <- sender clock + p.dormancy_h (after the first hop)
            FOR each tranche:
                t <- t + max(delay(mean_delay_h, delay_jitter), MIN_GAP)   # no identical times
                t <- snap_to_business_hours(t, p.business_hours_bias)
                emit transaction(sender, receiver, tranche, legs, pick_format(p.format_mix), t)
                next_holdings[receiver] += received amount
        holdings <- next_holdings
    IF template is cycle: emit a small return transfer from dst to src
    RETURN transactions        # relative genes only: total always equals the goal
```

Algorithm 4: Fitness of a candidate strategy

```
FUNCTION fitness(genome, episodes, detector, graph, tau):
    FOR each episode e:
        sub <- decode(repair(genome), e)
        scores <- detector.score(featurize(sub, overlay(graph, sub)))
        evasion <- 1 - (value of transactions with score >= tau) / (total value)
        completion <- delivered(sub, e) / e.goal
        cost <- fees / e.goal + lambda * elapsed / e.deadline
        f_e <- 0 IF completion < 0.95 ELSE max(completion^a * evasion^b - gamma * cost, 0)
    RETURN mean over episodes of f_e
```

Algorithm 5: The closed loop

```
FUNCTION run_arena(ledger, holdout, D0, rounds = 5):
    ASSERT holdout contains no adversary rows;  fp <- fingerprint(holdout)
    graph <- AccountGraph.build(ledger)        # built once; background never changes
```

```
    FOR r = 0 .. rounds - 1:
        A_r <- GA.evolve(fitness, D_r, graph, tau_r, seeds = recent pool genomes)
        strategies <- up to 8 distinct genomes of A_r with fitness > 0
        pool.add(strategies)
        rows <- pool.materialise(cap = 20% of original positives,
                                 equal row slices per round, time-window filters)
        D_{r+1} <- fit(original training rows + rows, same fixed hyper-parameters)
        tau_{r+1} <- alert_threshold(D_{r+1}.scores(holdout))
    cross_play[i][j] <- recall of D_i vs champion A_j (30 fixed episodes, fixed decode seed)
    ASSERT fingerprint(holdout) == fp
    RETURN curves, cross_play, per-round metrics
```

Algorithm 6: Real-data rules for the adversary

```
CLASS RealWorld(ledger):                    # used only when the ledger is real data
    currency <- US Dollar
    accounts <- accounts whose every transaction is in that currency
    formats  <- payment formats observed in the ledger (scrutiny-ordered)
    legs(amounts) -> same currency on both sides, received == paid
EpisodeSampler draws source, destination and mules only from RealWorld.accounts
# An operation may therefore do only what the data itself does:
# no currency conversion (which could create money) and no format the world never uses.
```

6.2 Implementation Details

The implementation is about 10,000 lines of Python (5,100 in the library, 1,300 in scripts, 1,900 in the dashboard and 1,600 in tests), and the following decisions shaped it.

- **A frozen graph with copy-on-write overlays.** A genetic-algorithm run needs about 20,000 candidate evaluations. Re-featurizing a multi-million-row ledger for each would be impossible, so the background graph is built once and each candidate sees a layered view that rebuilds only the few dozen accounts it touches. On the 422,648-account IBM graph, copying the dictionaries per candidate had cost tens of milliseconds per episode; layering them with a chain of maps brought one candidate evaluation from about 0.58 s to about 0.15 s.
- **One thread for small scoring batches.** The adversary scores a few dozen rows at a time. Giving each tiny batch to a many-thread prediction call made two concurrent runs contend for the CPU, so batches under 2,000 rows are scored on one thread with identical predictions.
- **As-of features.** Window queries use a strict left-sided search so a transaction never sees events at its own timestamp. This also means a strategy that emits all transfers at one instant would be invisible to burst features, which the decoder prevents with a minimum gap between transfers.
- **Time-based splits only.** The ledger is split 60/15/25 chronologically into training, validation and the frozen holdout; a random split would let the detector see an account's future.
- **Configuration overlay.** A YAML file holds every tunable. Setting ARENA_CONFIG to `config/ibm.yaml` re-points the same scripts at the IBM data and writes to separate folders, so the real-data results come from exactly the code that produced the synthetic ones.
- **IBM loader.** It parses the patterns file to attach a typology and pattern identifier to each

laundering transaction, drops self-transfers, cuts the days after background traffic stops, and labels positives that the patterns file does not list as unattributed. Every count removed is recorded in the data profile.

- **Reproducibility.** Seeds are fixed, every comparison uses the same episodes and decode seed, and each script prints a pass or fail gate against explicit thresholds.

Chapter 7

TESTING

7.1 Testing Strategy

Testing has two parts. **Automated tests** check that the code behaves as designed, above all that each of the four invariants holds. **Experimental evaluation** measures the detector and adversary on held-out data. The two meet in one rule: every flaw that experiments exposed was turned into an automated regression test, so it cannot return unnoticed.

7.2 Automated Test Suite

The suite contains 102 tests and all of them pass. Tests use small synthetic fixtures built by the same generator as the real pipeline, so they cannot drift out of step with the schema.

Table 7.1: Automated tests (102 in total)

Test file	Tests	What it guards
test_data.py	18	Schema validation and rejection of bad ledgers; label consistency; generator realism (FX-loss, amount and timestamp distributions of benign and laundering rows overlap); reproducibility
test_features.py	9	As-of correctness; batch path equals overlay path (invariant 1)
test_detector.py	19	The threshold spends exactly the alert budget, also with tied scores (invariant 3); metrics; chronological split; the holdout contains no adversary rows (invariant 4); downsampling; ranking quality; model save and load
test_adversary.py	20	Genome bounds; money conserved under crossover (invariant 2); plausibility floors; no account sends twice at the same instant; no currency or FX fingerprint on adversary rows; deadline handling; fitness rules (zero completion, completion floor, amount-weighted evasion)
test_loop.py	18	Injection cap, equal row slices and no starvation, time windows, per-episode goals, warm start, bootstrap intervals, end-to-end closed loop
test_explain.py	9	TreeSHAP contributions sum to the raw score, fixed and outlier-robust projection, flow layout, flows scored by both judges
test_ibm.py	9	IBM cleaning (self-transfers, dead tail, unattributed), real-data rules, money conservation over 300 random genomes, configuration overlay

Several tests were written because an earlier version of the experiment had failed. For instance, the loop tests include a regression for the case in which counting adversary operations instead of rows let one round starve the others, and the adversary tests fingerprint the decoder's output so that no constant, such as a fixed fee, can single out adversary rows.

7.3 Experimental Evaluation

Protocol

All splits are chronological. Every metric is computed on the **frozen clean holdout**, the last 25% of the

ledger in time, at its true prevalence. The primary metric is **recall at a 0.5% alert budget**; PR-AUC is the threshold-free check, and chance level for PR-AUC equals the prevalence (0.12% on IBM). Accuracy is never reported and ROC-AUC is reported only with a warning. Adversary results are measured on fixed held-out episodes from a later period than any episode used for training.

Phase 1: the detector, synthetic and real data

Table 7.2: Detector results on the frozen clean holdout

Frozen clean holdout	Synthetic	IBM HI-Small
Transactions / laundering	500,001 / 1,132	1,121,605 / 1,324
Recall at 0.5% budget	0.537	0.668
PR-AUC	0.454	0.424
Precision at budget	0.243	0.158
ROC-AUC (misleading at this prevalence)	0.971	0.983
Trees	205	395
Phase 1 gate (PR-AUC >= 0.40, recall >= 0.50)	PASS	PASS

On IBM data the cleaned ledger has 4,486,418 transactions, 4,514 of them laundering (0.101%), over 422,648 accounts and 10 days. The detector's recall is 0.668 and its PR-AUC 0.424, about 360 times the chance level of 0.0012. Fig. 7.1 shows the precision-recall curve with the operating point.

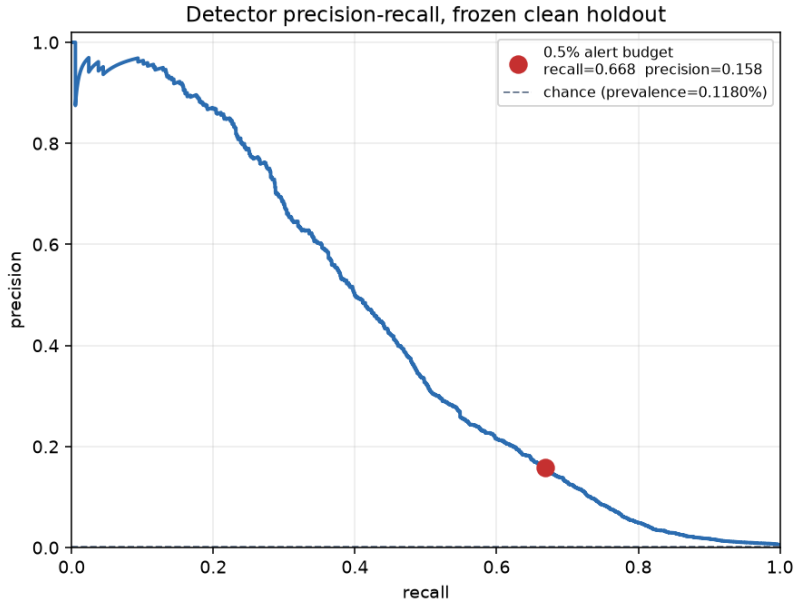


Fig. 7.1: IBM HI-Small: precision-recall curve on the frozen holdout, with the 0.5% operating point

Table 7.3: Recall at the alert budget by laundering typology

Typology	Synthetic recall	IBM recall
Fan-out	0.414	0.906
Fan-in	0.411	0.952
Cycle	0.481	0.937
Gather-scatter	0.458	0.956
Stack	0.745	0.863
Unattributed (IBM only)	n/a	0.255

The labelled typologies are caught at 0.86 to 0.96 on IBM data. The 44% of IBM positives that the patterns file does not list are caught at only 0.26, so the headline recall averages two very different populations.

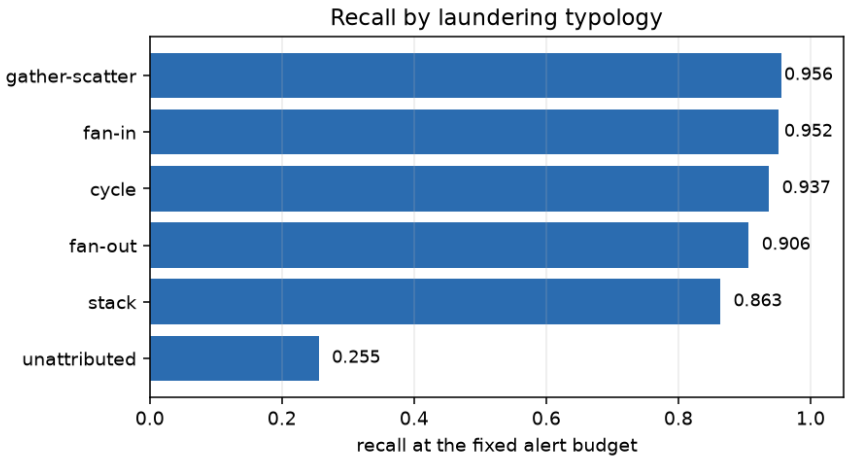


Fig. 7.2: IBM HI-Small: recall at the alert budget by typology

Phase 2: the adversary on synthetic data

Table 7.4: Synthetic data: the evolved strategy against the detector

Strategy	Detector recall on its transactions	Money delivered
Naive, hand-written (3 transfers)	0.750	99.9%
Random genomes (mean)	0.748	n/a
Evolved by the GA (stack, 8 tranches, threshold hug 0.67)	0.231	99.9%

The genetic algorithm cut the detector's recall by 52 points while still delivering essentially all of the money. Random, unevolved genomes were no better than the naive strategy (0.748), so the gain comes from selection, not from the genome space itself.

Phase 3: the arms race on synthetic data

Table 7.5: Synthetic data: the arms race over five rounds (200 held-out episodes)

Round	Detector recall against its adversary (95% interval)	Adversary success	Clean recall / PR-AUC
0	0.125 (0.11 to 0.14)	99%	0.538 / 0.454
1	0.275 (0.22 to 0.34)	72%	0.519 / 0.437
2	0.295 (0.24 to 0.36)	70%	0.529 / 0.441
3	0.365 (0.30 to 0.44)	64%	0.516 / 0.427
4	0.510 (0.48 to 0.54)	56%	0.523 / 0.430
Final D_5			0.509 / 0.421

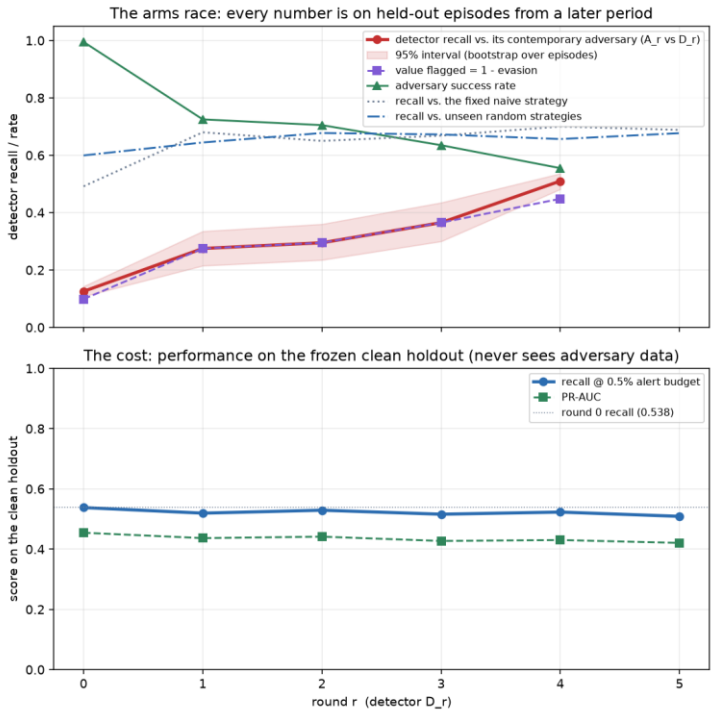


Fig. 7.3: Synthetic data: detector recall and adversary success over five rounds

Feedback improved the detector against the strategies it was shown in every round, at a cost of less than three points of clean recall. It did not get ahead: after five rounds the adversary still succeeds in 56% of episodes. The cross-play matrix (Fig. 7.4) shows the adversary alternating between two families, a multi-hop fan-out and a single large transfer, rather than escalating.

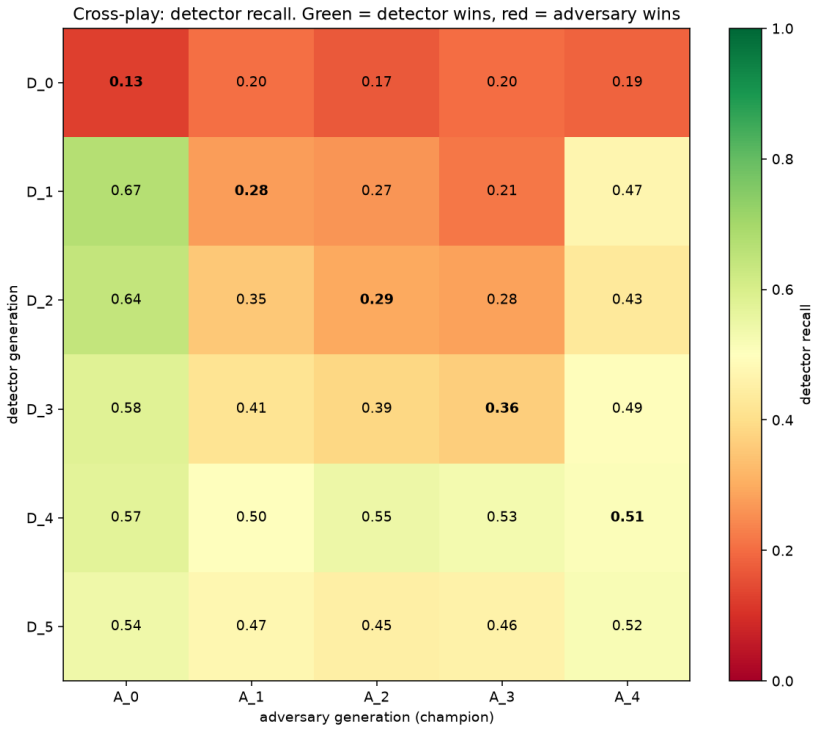


Fig. 7.4: Synthetic data: cross-play matrix, every detector against every adversary

Phase 4: attribution and decisions

For every detector the mean absolute TreeSHAP contribution of each feature was computed on the adversary's transactions, the holdout laundering and holdout benign traffic (Fig. 7.5 shows the adversary's operations). On those operations `amount_log` rose from 0.85 to 1.70 and `snd_out_vol_24h` from 0.18 to 0.38, while `snd_age_h` fell from 0.44 to 0.28 and `threshold_gap` from 0.42 to 0.30: the detectors learned that the adversary's large transfers are suspicious and that account age and distance from the threshold no longer separate it. `amount_mod_100` also rose (0.59 to 0.75); that reflects an artefact of the synthetic generator (Section 7.4), not a discovery.

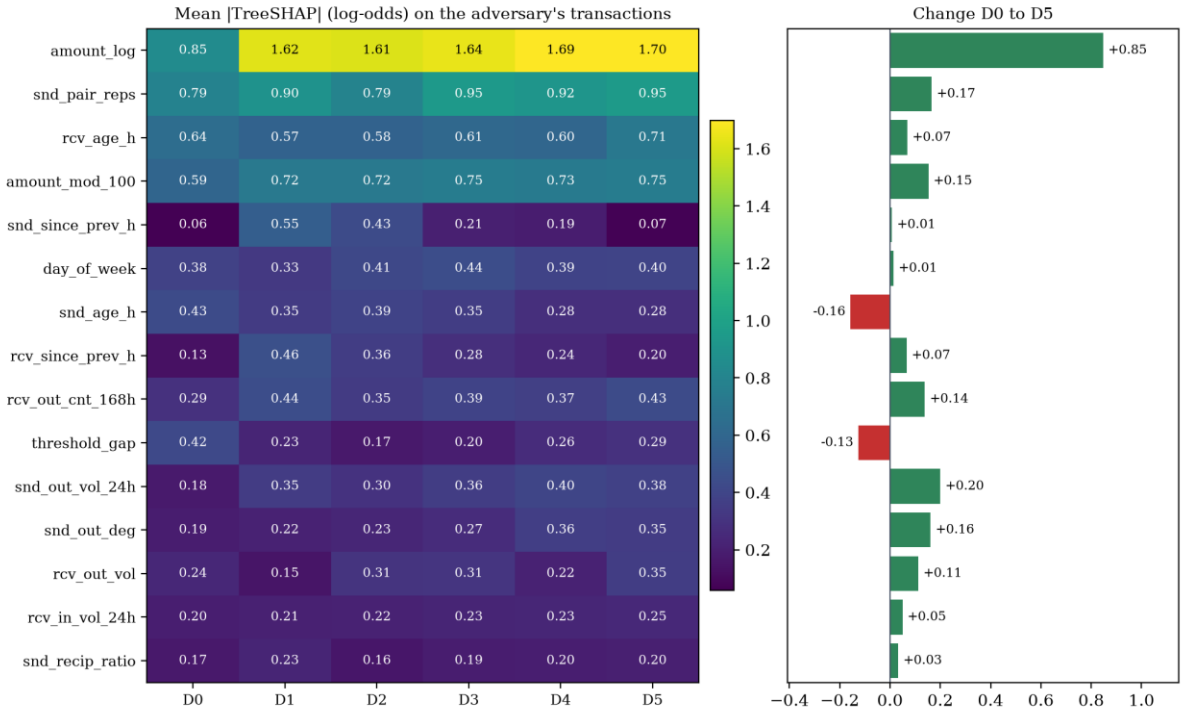


Fig. 7.5: Mean absolute TreeSHAP attribution per feature on the adversary's transactions, detectors D_0 to D_5

Fig. 7.6 places 3,000 holdout benign, 1,132 holdout laundering and 1,023 adversary transactions in one fixed two-component plane and colours them by each detector's decision. D_0 flags 17.4% of the adversary's transactions and D_1 flags 54.5%; after that the picture barely changes, matching the plateau in the cross-play matrix. The plane explains 42.6% of the variance and is a map of where decisions fall, not a picture of the model.

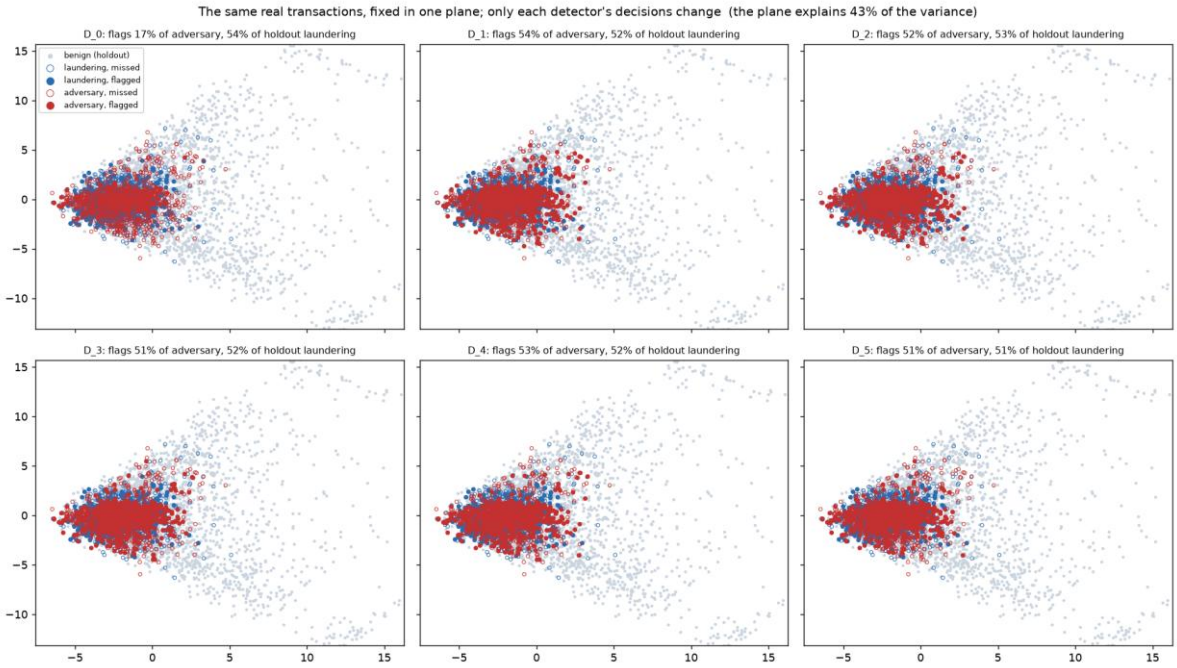


Fig. 7.6: Fixed projection of real transactions coloured by each detector's decisions

Phase 5: real IBM data

Transfer between the two worlds. Each detector was scored on the other dataset's frozen clean holdout, applying the same alert-budget policy to the target's own traffic.

Table 7.6: Cross-dataset transfer

Recall at budget (PR-AUC)	Scored on synthetic	Scored on IBM
Trained on synthetic	0.537 (0.454)	0.000 (0.001)
Trained on IBM	0.016 (0.003)	0.668 (0.424)

Chance recall at a 0.5% budget is about 0.005, so neither detector recognises the other world's laundering. The synthetic numbers are therefore a check that the pipeline works, and the IBM numbers are the result.

Feature groups. PaySim, the secondary dataset named in the synopsis, has no comparable account graph, so only the 17 per-transaction features would carry over. Retraining on each feature group shows what such a validation would have kept.

Table 7.7: Retraining on one feature group at a time

Recall at budget (PR-AUC)	Synthetic	IBM
All 74 features	0.537 (0.449)	0.668 (0.424)
Per-transaction only (17)	0.155 (0.023)	0.187 (0.019)
Relational only (57)	0.455 (0.366)	0.617 (0.290)
All but payment format (72)	0.527 (0.444)	0.636 (0.351)

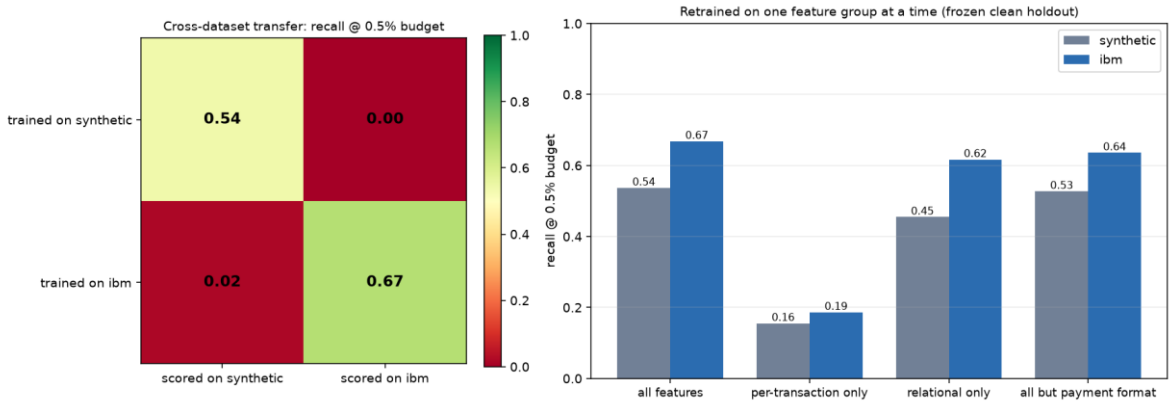


Fig. 7.7: Cross-dataset transfer (left) and feature-group ablation (right)

The adversary on IBM data. The genetic algorithm reaches a fitness of 1.000, but so does the hand-written naive strategy (0.998). The detector's recall on the naive strategy is 0.000, on random genomes 0.006 and on the evolved champion 0.000. The cause is a dataset shortcut: 88% of IBM laundering is paid by ACH against 12% of benign traffic, and the IBM detector leans on it (36% of its gain). Changing only the payment format of the naive strategy's transfers to ACH raises the detector's median score from 0.0006 to 0.62. Retraining without the two payment-format features (60 held-out episodes) gives:

Table 7.8: The effect of removing the payment-format shortcut

Strategy	IBM detector: recall / adversary wins	Retrained without payment format
Naive	0.000 / 100%	0.167 / 100%
Naive, paid by ACH	0.144 / 100%	0.167 / 100%
Phase 2 champion	0.000 / 100%	0.233 / 78%
Clean holdout recall (PR-AUC)	0.668 (0.424)	0.636 (0.351)

Removing the shortcut costs three points of recall on IBM's own laundering and makes the detector measurably harder to evade, but it is still evaded in most episodes.

The closed loop on IBM data used the same settings as the synthetic run, with 30 held-out episodes per cell and a 24-hour deadline because HI-Small spans only ten days.

Table 7.9: IBM HI-Small: the arms race over five rounds

Round	Clean recall / PR-AUC	Detector recall against its adversary	Adversary success
0	0.668 / 0.424	0.000	100%
1	0.660 / 0.420	0.200	80%
2	0.664 / 0.390	0.067	93%
3	0.657 / 0.407	0.125	100%
4	0.668 / 0.399	0.033	97%
Final D_5	0.669 / 0.382		

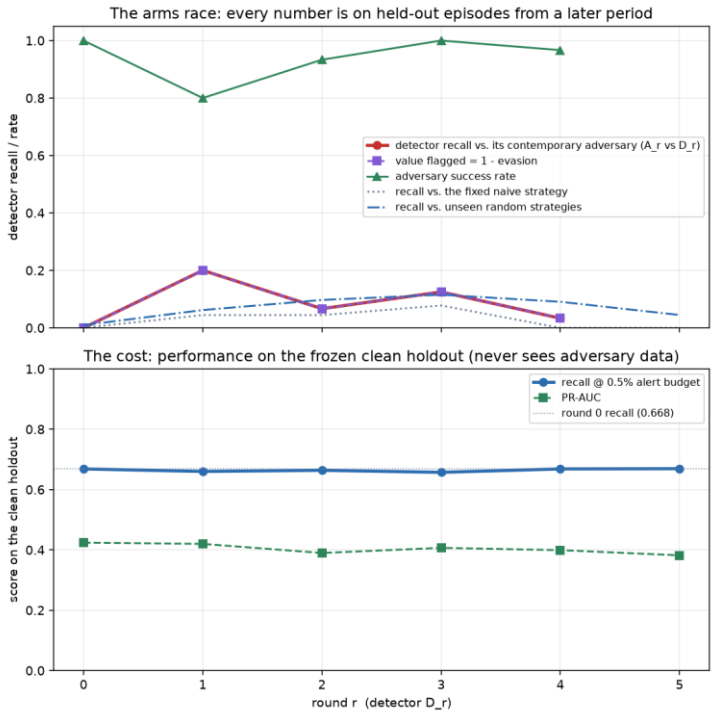


Fig. 7.8: IBM HI-Small: detector recall and adversary success over five rounds

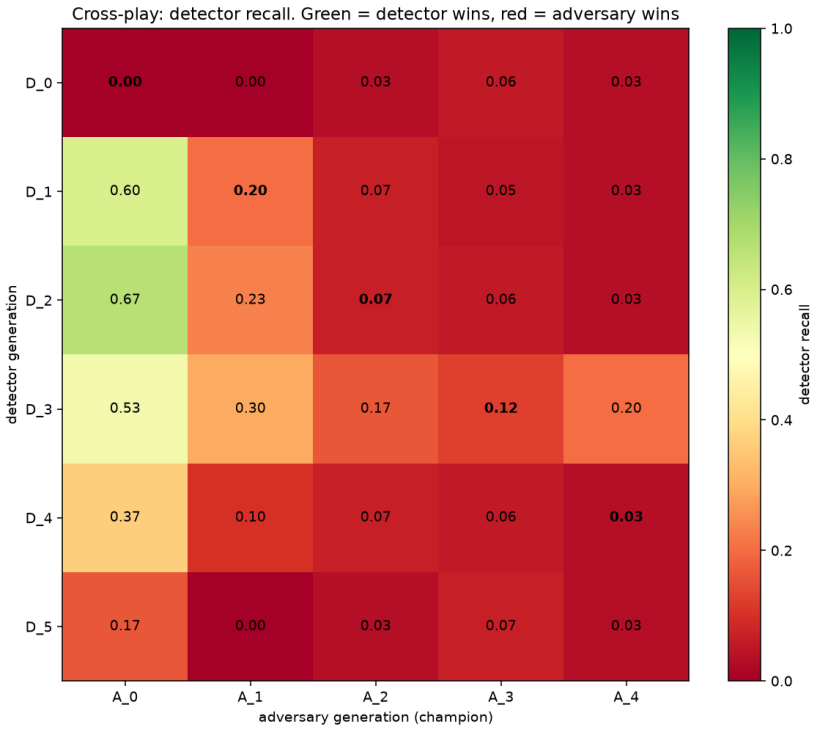


Fig. 7.9: IBM HI-Small: cross-play matrix

The loop's guards held: the holdout was unchanged, the alert budget was exact in every round, feedback stayed within its 20% cap, and the Phase 3 gate passed. But feedback did **not** help. Each retrained detector learns the strategy it was shown (D_1 catches 0.60 of A_0) and then loses to the next, which finds another way through, and clean-holdout PR-AUC falls from 0.424 to 0.382. On synthetic data the same loop raised recall from 0.125 to 0.510; on IBM it stays between 0.00 and 0.20. With 30 episodes per cell and one seed the figures are indicative, not tight, but the direction is clear: a detector that leans on a dataset shortcut cannot be repaired by patching individual strategies.

7.4 Defects Found by the Adversary and the Experiments

The most important testing result is a list of flaws in the data, simulation and experiment that an ordinary evaluation had passed. In each case the adversary reported an implausibly good score, the decoded transactions were read, and the cause was fixed structurally, by making two paths share code rather than by correcting the output.

Table 7.10: Flaws exposed by the adversary

Stage	Symptom	Cause	Fix
Phase 1	Detector PR-AUC 1.000	Three generator leaks: an FX-loss band unique to laundering, laundering amounts above the benign range, laundering timestamps uniform across the clock	Benign and laundering rows routed through shared functions
Phase 2	Detector recall 0.000	Twelve transfers at one identical timestamp, invisible to as-of features	Minimum gap between transfers in the decoder
Phase 3, run 1	Recall 0.000 in round 1	One $250,000.00 transfer: a fixed round goal, while synthetic laundering is never round	Per-episode goals drawn log-uniformly to the cent
Phase 3, run 2	Retrained detector too good	Every adversary row had FX loss exactly 0.002 (a fixed fee)	Fee set to 0; currency legs from the world's own process
Phase 3, run 3	GA fitness 0.999, held-out far lower	Winner's curse: a one-transaction strategy scores five out of five by luck	200-episode re-evaluation with bootstrap intervals
Phase 5	IBM evasion 1.000, completion 525 times the goal	Chained currency conversions created money; one account paid in five currencies; a format that IBM has only as a self-transfer	RealWorld rules: one currency, only observed accounts and formats; regression test

7.5 Discussion

The results support the central premise of the project. A detector with 0.668 recall on a held-out IBM test set catches none of the transfers of a simple strategy paid by cheque instead of ACH. A static score describes the dataset's signature as much as the detector's ability. The arms race also shows what retraining can and cannot do: on synthetic data it raised recall against the adversary from 0.125 to 0.510 at little cost, but on IBM data, where the weakness is a dataset shortcut and not a strategy, it did not help, whereas removing the shortcut helped more.

The limits of the evidence are stated plainly. The IBM data is itself simulated; the arms race is one seed per run, with intervals over episodes and not over repeated runs; the adversary reads the detector's exact scores, which a real launderer cannot, so it is stronger than a realistic attacker; and PaySim was not used. Synthetic laundering in the generator is never a round amount while 21.6% of benign amounts are, which is worth about three points of Phase 1 recall; the artefact is absent from the IBM data (round amounts are 0.013% of benign and 0.022% of laundering).

Chapter 8

SCREENSHOTS

The dashboard is a Streamlit application with nine pages: an overview, one page for each of the five phases, a live strategy explorer, an architecture walkthrough and a limitations page. Apart from the live explorer, every page replays the stored results of a finished run and never retrains or re-evolves anything. Run it with `uv run streamlit run dashboard/app.py`.

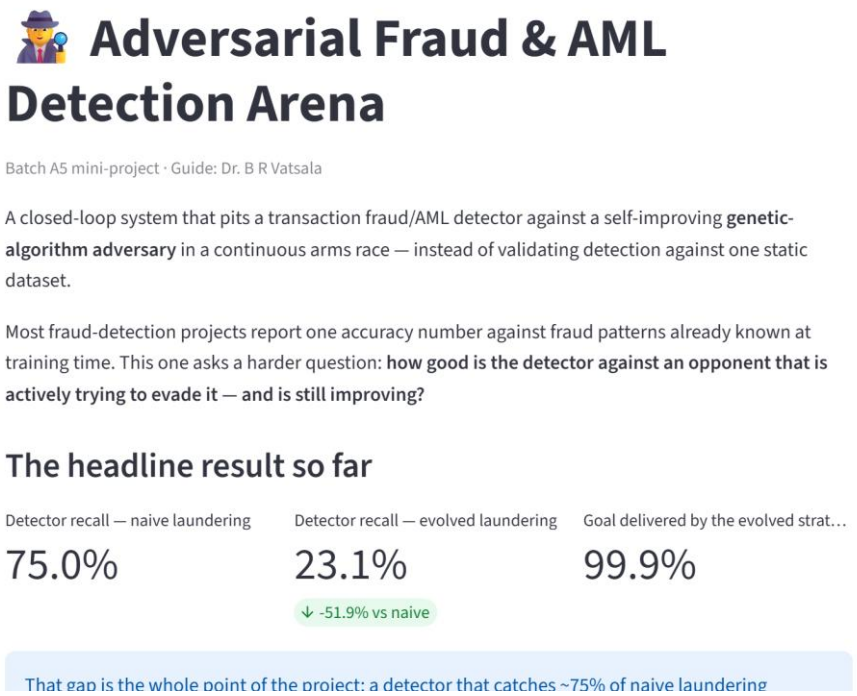


Fig. 8.1: Overview page: the headline result and the four invariants

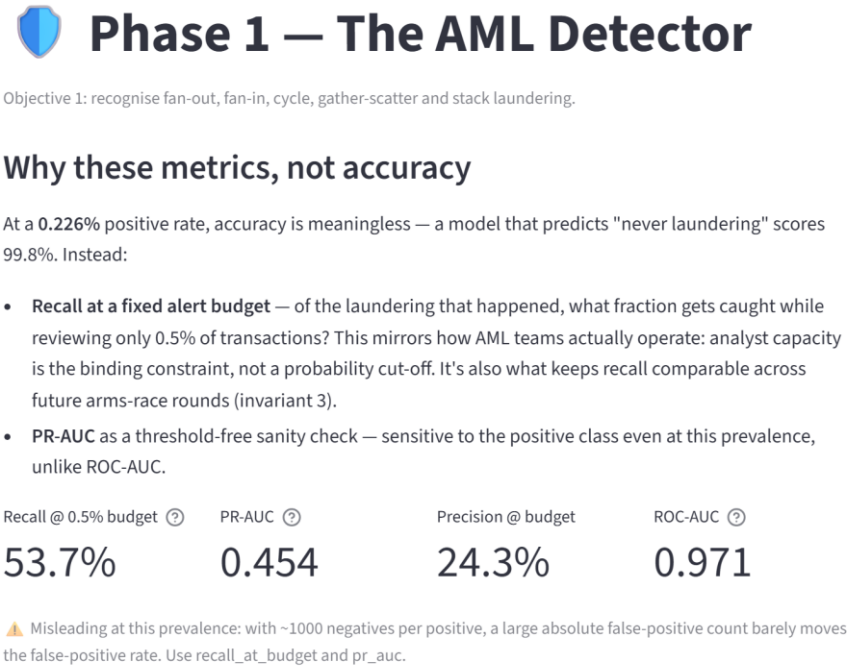


Fig. 8.2: Phase 1 page: the detector and why accuracy is not used

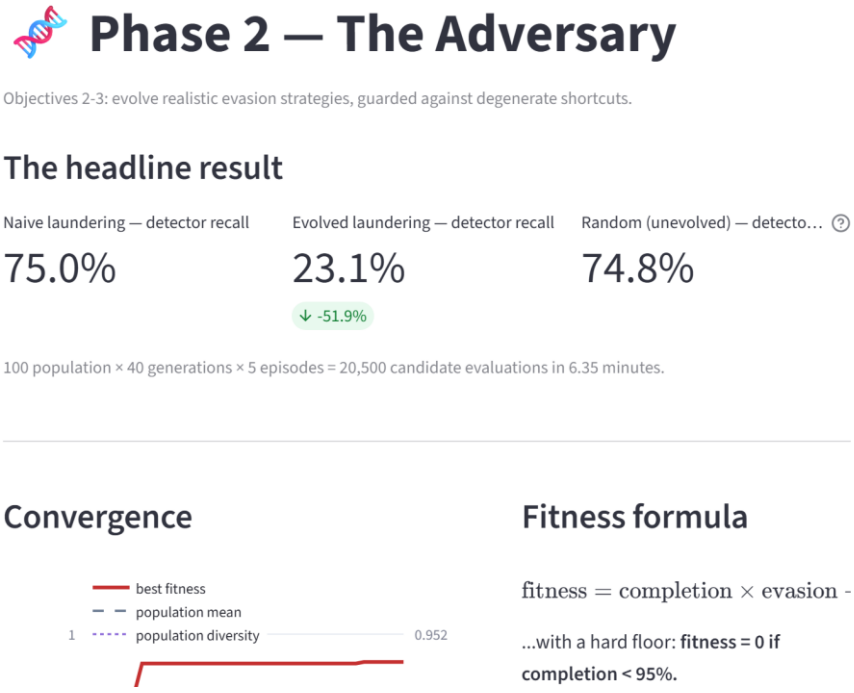


Fig. 8.3: Phase 2 page: genetic-algorithm convergence and the naive versus evolved gap

Phase 3 — The Arms Race

Objective 4: feed the adversary's evasive strategies back into the detector, retrain, and measure recall against the adversary across rounds.

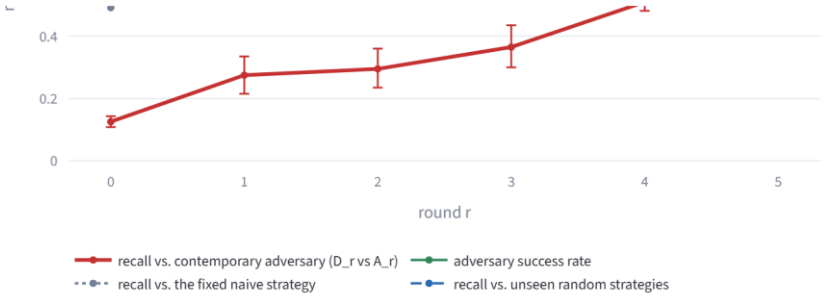
One **round** is a fight and a lesson: the current detector **D_r** is attacked by an adversary **A_r** evolved against it, the adversary's best strategies are fed back into the detector's training data, and the detector is retrained into **D_(r+1)**. This run: **5 rounds**, so 6 detectors and 5 adversaries, in 29 minutes.

Detector recall vs. its contempo…	Adversary success rate	Clean-holdout recall @ 0.5% bu…
51.0%	56%	50.9%
↑ +38.5% since round 0	↓ -44% since round 0	↓ -2.9% since round 0

The arms-race curve

Every line is measured on the same fixed episodes, drawn from a later period than any the detector trained on, so a change between rounds means the detector or adversary changed — not the dice. **200 episodes per point; the red bars are 95% bootstrap intervals over episodes.** A strategy that sends one transaction is a coin flip per episode, so with 30 episodes the error bar is wider than most round-to-round changes — which is why this was re-measured.

Fig. 8.4: Phase 3 page: the arms-race curve with bootstrap intervals



Red answers *can the adversary still win?* **Grey and blue** are the generalisation probes: strategies the detector was never shown. If feedback only taught it to recognise the exact strategies it saw, these stay flat; if it learned something general, they rise.

Cross-play: every detector against every adversary



Fig. 8.5: Phase 3 page: cross-play matrix and the cost on the clean holdout

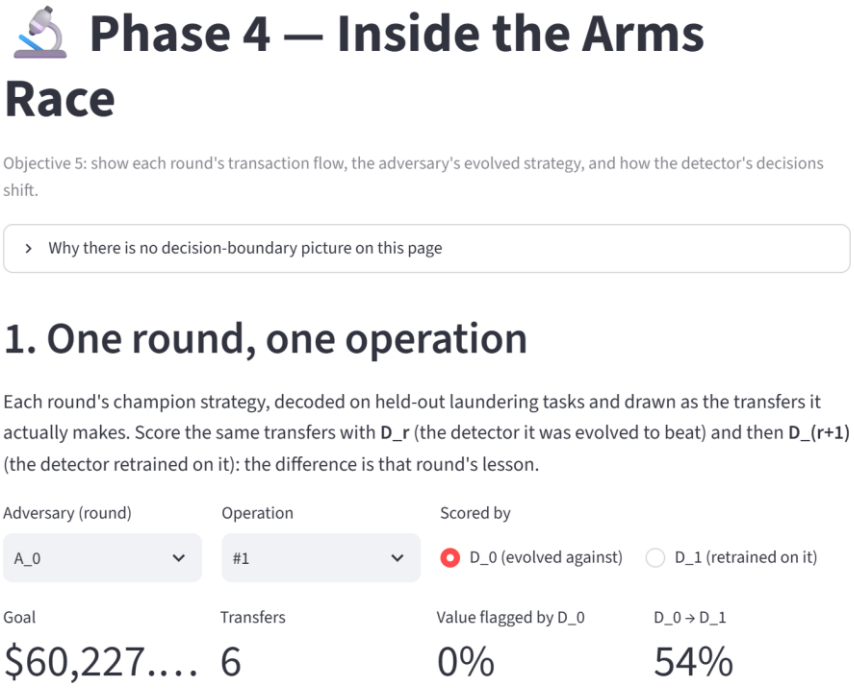
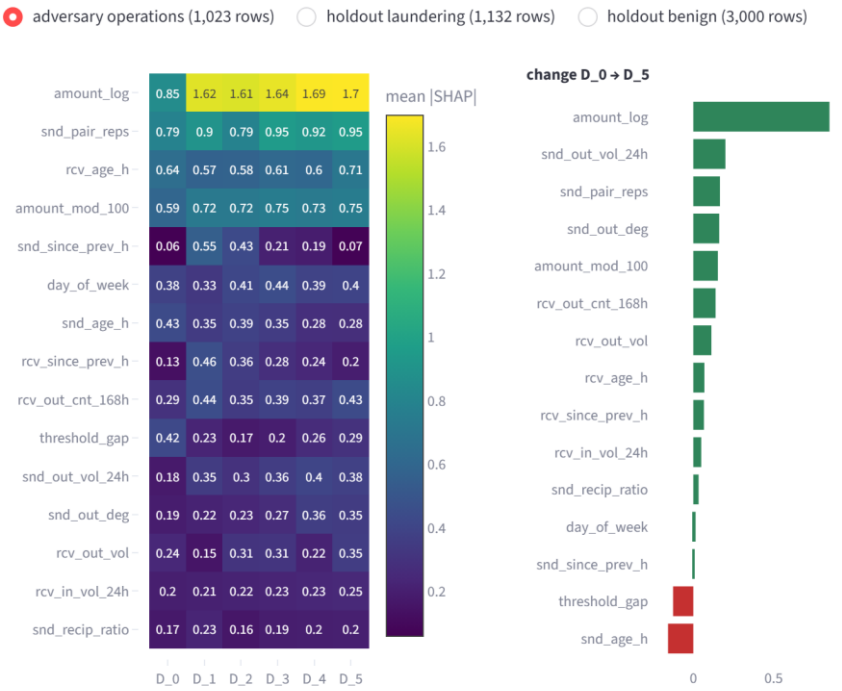


Fig. 8.6: Phase 4 page: one round, one operation: decoded transfers scored by D_r and $D_{(r+1)}$



- `amount_mod_100` **rises.** That is the generator's round-number artefact (synthetic laundering is never

Fig. 8.7: Phase 4 page: attribution drift per feature per detector

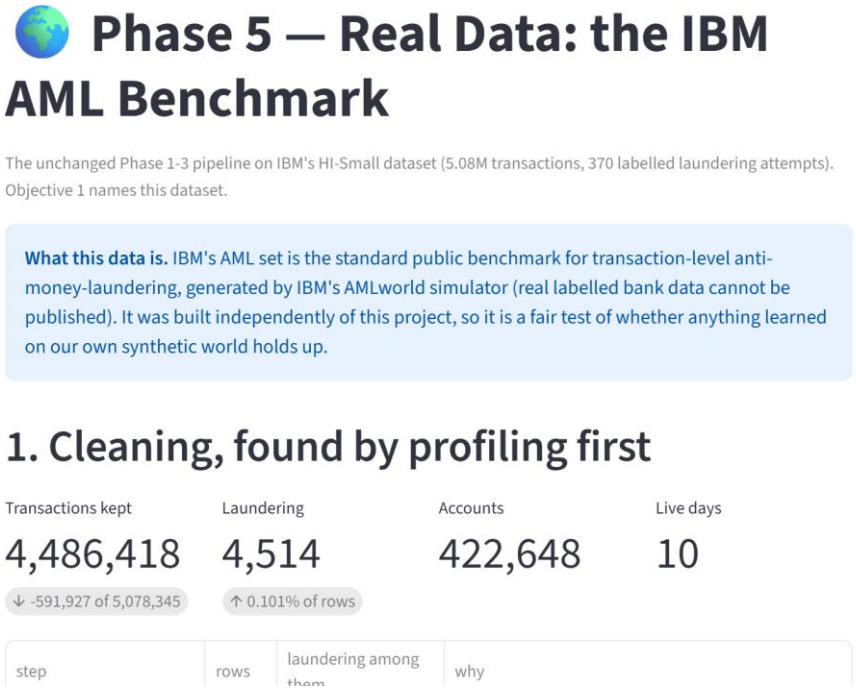


Fig. 8.8: Phase 5 page: IBM data cleaning, found by profiling the dataset first

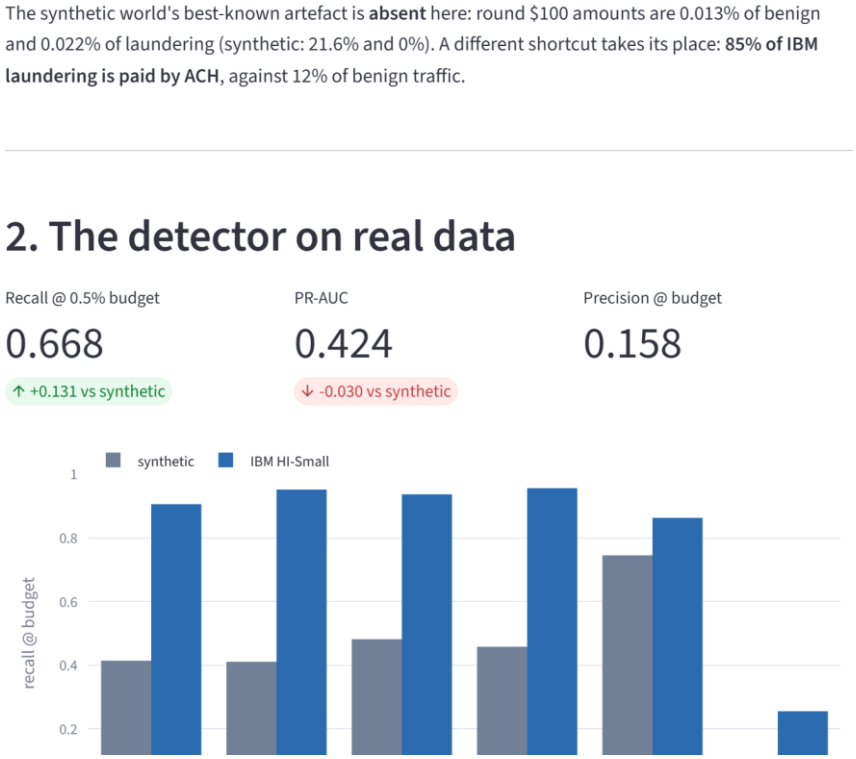


Fig. 8.9: Phase 5 page: the detector on real data, recall by typology

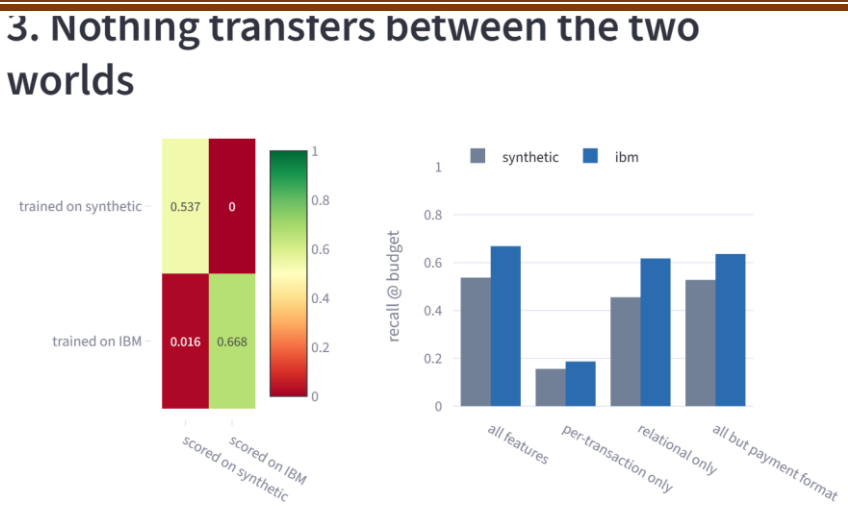


Fig. 8.10: Phase 5 page: transfer between the synthetic and the IBM world

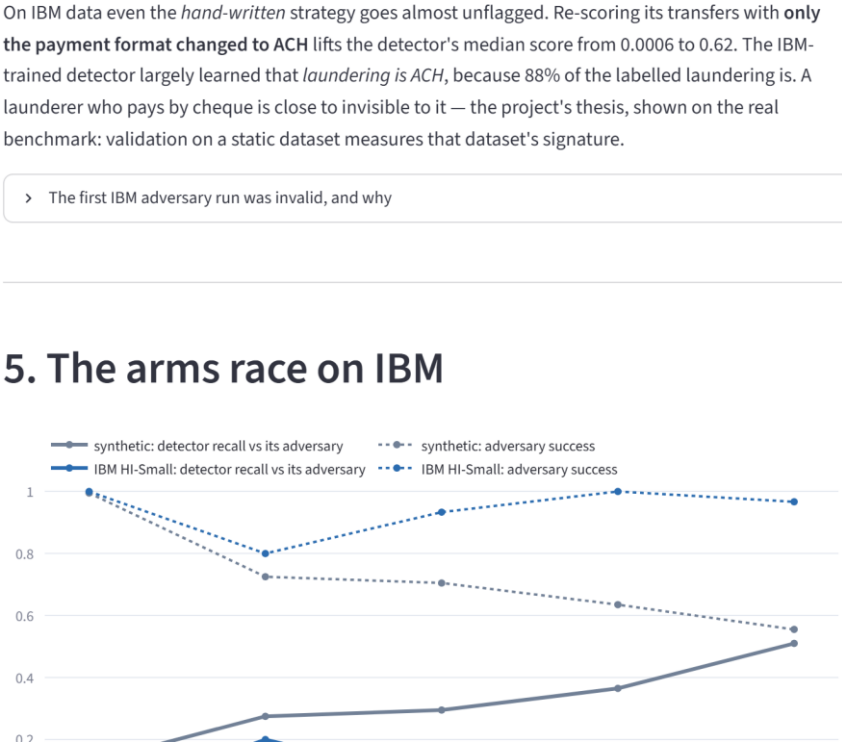


Fig. 8.11: Phase 5 page: why the adversary wins on IBM data, and the arms race beside the synthetic one



Fig. 8.12: Phase 5 page: IBM holdout cost and the cross-play matrix

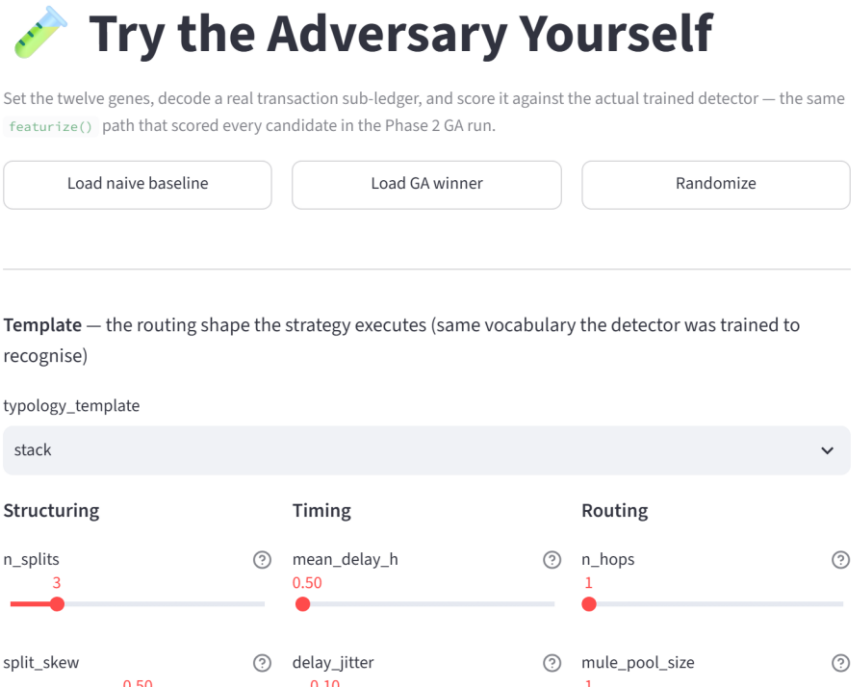


Fig. 8.13: Live explorer: set the twelve genes and score the decoded transactions

FUTURE ENHANCEMENT AND CONCLUSION

Future Enhancements

- **A shortcut-resistant detector inside the arms race.** The format-blind detector was only evaluated against fixed strategies; running it through the closed loop would show whether removing the shortcut lets feedback work on real data.
- **Repeated runs.** Repeating the closed loop over several seeds would give intervals over runs, not only over episodes, and a firmer reading of the IBM arms race.
- **A realistic attacker.** An adversary that sees only the detector's decisions (flag or no flag), not its scores, would be closer to a real launderer and would show how much the current results overstate the attacker.
- **More data and models.** The larger IBM variants (HI-Medium, LI-Small), PaySim for the per-transaction features, and graph neural network detectors compared with the tree model.
- **Richer feedback and a live feed.** Negative examples for strategies that failed, and a streaming simulation of transactions in place of the replay.

Conclusion

The Adversarial Fraud & AML Detection Arena does what the synopsis set out to do. It measures an AML detector against an opponent that adapts, round by round, and presents the result as an observable process instead of a single accuracy figure. A LightGBM detector was trained on the IBM AML dataset and recognises its laundering typologies at a recall of 0.668 at a 0.5% alert budget; a genetic-algorithm adversary with a twelve-gene genome evolves structuring, timing and routing strategies that are decoded into valid, money-conserving transactions; a fitness function balances evasion against completion; a closed loop feeds the strategies back and measures detector recall and adversary success over five rounds; and a dashboard shows the flows, strategies and the detector's response.

Three findings stand out. First, static scores overstate robustness: on IBM data a detector with recall 0.668 caught none of the transfers of a simple strategy paid by cheque. Second, feedback helps only when the weakness is a strategy: on synthetic data it raised recall against the adversary from 0.125 to 0.510, while on IBM data, where the weakness is a dataset shortcut, it did not. Third, an adversary is an effective auditor: its implausibly good scores exposed six flaws in the data, simulation and experiment, each of which an ordinary evaluation had passed. The limitations are stated in Section 7.5.

REFERENCES

[1] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, 2002.

[2] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, 2018.

[3] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, 2015.

[4] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symposium Series on Computational Intelligence (SSCI)*, 2015.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[6] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.

[7] M. Weber et al., "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019.

[8] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th European Modeling and Simulation Symposium (EMSS)*, 2016.

[9] E. Altman, J. Blanusa, L. von Niederhausern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

[10] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[11] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial machine learning," *Pattern Recognition*, vol. 84, 2018.

[12] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975.

[13] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagne, "DEAP:

Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, 2012.

[14] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, 2020.

[15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, 2011.

[16] E. Altman, "IBM Transactions for Anti Money Laundering (AML)," Kaggle dataset, kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering (HI-Small variant used).